



Tecnológico de Monterrey

Instituto Tecnológico y de Estudios Superiores de Monterrey
Campus Puebla

TE3002B: Implementación de robótica inteligente (Gpo 501)

Profesor: Dr. Rigoberto Cerino Jiménez

Reto semanal 4. Manchester Robotics

Equipo *I miss NWJNS*

Tania Sofia Arrazola Bello	A01738124
Iñaki Ahedo Madrid	A01738231
Marcos Allen Martínez Cortés	A01737939

11 de mayo de 2026

Resumen

En el presente *Half Term Challenge*, se desarrolló un sistema de navegación autónoma para el robot Puzzlebot bajo el framework ROS 2, destacando la integración de una capa de toma de decisiones basada en visión computacional. La arquitectura distribuida permite la localización mediante odometría y el seguimiento de trayectorias punto a punto, administradas por el tópico `/goals` y ejecutadas a través de comandos de velocidad en `/cmd_vel`. El nodo de principal de visión artificial es capaz de interpretar tres estados críticos: Rojo (paro total), Amarillo (reducción de velocidad al 70%) y Verde (marcha normal).

El sistema de visión emplea una cámara embebida y procesamiento con OpenCV, utilizando el espacio de color HSV y segmentación por máscaras binarias. Se implementó un análisis geométrico de anillo para correlacionar el núcleo brillante de la luz con su halo cromático, mitigando falsos positivos por ruido ambiental. La detección se consolida mediante un *SimpleBlobDetector*, optimizado con filtros de área y una circularidad mínima de 0.72 para garantizar precisión a una distancia de 30 cm.

En el ámbito de control, se diseñó una arquitectura de doble lazo cerrado: un lazo externo con un controlador PID sintonizado para la velocidad lineal, y un lazo interno que regula la potencia de los motores mediante la retroalimentación de la odometría. Para el control angular, se empleó un esquema proporcional apoyado en funciones como *Wrap to Pi* y conversiones de cuaterniones a ángulos de Euler. Finalmente, una máquina de estados gestiona la seguridad del sistema; ante una luz roja, el estado *waiting_for_green* bloquea el movimiento, asegurando que el robot no reanude su trayectoria hasta recibir una señal verde explícita, incluso ante oclusiones temporales del sensor.

Objetivos

El objetivo principal de este proyecto es desarrollar un sistema de navegación multipunto en lazo cerrado para el robot Puzzlebot utilizando ROS 2. El sistema integra un controlador PID y un nodo de localización por odometría para ejecutar trayectorias complejas con alta precisión y robustez. Para alcanzar este propósito, se establecieron los siguientes objetivos específicos:

- **Localización y Estimación de Pose.** Implementar un nodo de localización que estime en tiempo real la pose del robot (x , y , θ) mediante la integración de la cinemática diferencial y el procesamiento de datos provenientes de los encoders.
- **Generación Dinámica de Trayectorias.** Diseñar un generador de trayectorias encargado de publicar secuencialmente los puntos de destino, validando la alcanzabilidad de cada meta según la dinámica del robot.
- **Control de Lazo Cerrado.** Desarrollar un controlador para regular las velocidades lineal y angular, garantizando el seguimiento preciso de los objetivos mediante la publicación de mensajes de tipo Twist en el tópico `/cmd_vel`.

- **Optimización y Robustez.** Sintonizar las ganancias del controlador (K_p , K_i , K_d) para minimizar el error en estado estacionario y aplicar estrategias que mitiguen el ruido de los sensores y las no linealidades del sistema.
- **Navegación Basada en Visión.** Desarrollar un sistema de visión computacional para la detección cromática de un semáforo, estableciendo una jerarquía de control que determine el comportamiento del robot: detención total (Rojo), velocidad reducida (Amarillo) o continuación de la trayectoria (Verde).
- **Gestión de Datos mediante ROS 2.** Utilizar mensajes personalizados para la administración eficiente de metas a través del tópico `/goals`, asegurando una comunicación robusta entre los nodos de visión, trayectoria y control.

Introducción

El presente reto aborda el diseño y la implementación de un sistema de navegación autónoma para un robot móvil basado en el framework ROS 2, fundamentado en un esquema de control en lazo cerrado y localización mediante odometría, además del reconocimiento de un semáforo por visión computacional. El objetivo central es la ejecución de una trayectoria determinada por el usuario, empleando un controlador PID sintonizado para corregir el desplazamiento del robot en tiempo real a partir de la estimación continua de su pose, y adecuándose al comportamiento determinado por el estado del semáforo.

Odometría de un robot móvil

La odometría es el estudio de la estimación de la posición y el trayecto de vehículos o robots móviles mediante datos de sensores de movimiento, su nombre proviene del griego *hodos* (“viajar”, “trayecto”) y *metron* (“medida”), y se basa en la integración de información incremental para calcular cambios respecto a un punto de inicio [1, 2]. Existen diferentes métodos de medición entre los que se encuentran los encoders de las ruedas, las unidades de medición inercial (IMUs) y la odometría óptica y visual [2].

La odometría destaca por su bajo costo, alta velocidad de muestreo y buena precisión a corto plazo, aunque presenta desafíos críticos como la acumulación de errores, estimación de una posición relativa y no absoluta, y sensibilidad a factores externos como el deslizamiento de las ruedas; debido a esto, se utiliza la fusión de sensores combinando diferentes métodos, aunque esto requiere algoritmos más complejos y mayor potencia de procesamiento [1, 2].

Un robot móvil diferencial se caracteriza por poseer dos ruedas unidas por un eje transversal, como se muestra en la figura 1, donde cada rueda es accionada de forma independiente por su propio motor y monitoreada por un encoder que registra las revoluciones del eje [1]. Esta configuración permite el desplazamiento y la rotación del sistema mediante la diferencia de velocidades de ambas ruedas. Para estimar la posición

del robot en el plano, se recurre al Método de Euler, el cual permite transformar las ecuaciones diferenciales de movimiento en un problema de condiciones iniciales (IVP) donde la posición actual $[x_n, y_n, \theta_n]$ actúa como el punto de partida para predecir el siguiente estado [3]. Al aplicar la aproximación de Euler, se asume que las velocidades lineal y angular permanecen constantes durante un intervalo de tiempo pequeño h o dt , permitiendo calcular la nueva postura como $y_{n+1} = y_n + h \cdot f_n$.

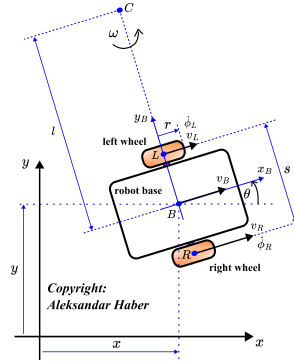


Figura 1: Robot móvil diferencial.

https://aleksandarhaber.com/wp-content/uploads/2023/10/robot_dd2.png

Cálculo del error en un robot móvil diferencial

En el cálculo de la odometría, el error se manifiesta principalmente en dos componentes: el error lineal y el error angular, los cuales se derivan de las discrepancias entre el movimiento real y la interpretación de los datos de los encoders. El error lineal se asocia frecuentemente con el parámetro E_d (diámetro de las ruedas), donde una diferencia entre los diámetros de la rueda derecha D_R y la izquierda D_L , provoca que el robot recorra una distancia distinta a la calculada; matemáticamente este se estima comparando la posición absoluta final (x_{abs}, y_{abs}) , contra la posición calculada (x_{calc}, y_{calc}) tras completar una trayectoria. Por otro lado, el error angular es sumamente crítico debido a que pequeñas desviaciones en la orientación se magnifican con la distancia recorrida; este error suele vincularse al parámetro E_b , que representa la incertidumbre en la distancia entre ejes del robot [1].

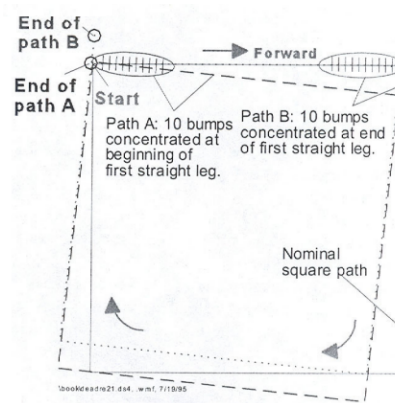


Figura 2: Prueba UMBmark [1].

Para calcular estos errores de manera efectiva, se utiliza la diferencia de retorno en pruebas de trayectoria cerrada, como la prueba UMBmark que se puede observar en la figura 2. En estas pruebas el error lineal se cuantifica mediante el desplazamiento en los ejes x e y ($\epsilon_x = x_{abs} - x_{calc}$ y $\epsilon_y = y_{abs} - y_{calc}$), mientras que el error angular se determina a través del error de orientación en retorno ($\epsilon_\theta = \theta_{abs} - \theta_{calc}$). Mientras que el error lineal puede ser causado por el deslizamiento o diámetros desiguales, el angular es particularmente sensible a la resolución de los encoders y a la anchura de la base del vehículo. La distinción entre ambos es vital, ya que los errores sistemáticos (relacionados al diseño y mecánica del robot) tienden a hacer que las elipses de incertidumbre, como se observan en la figura 3, crezcan de forma acelerada en la dirección de la marcha [1].

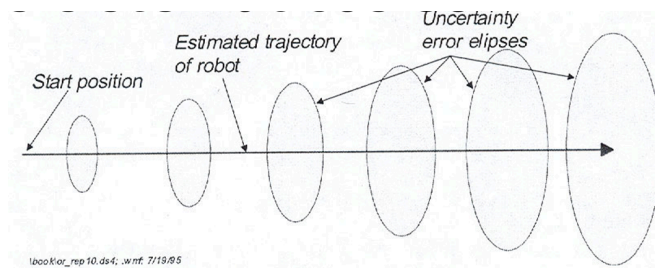


Figura 3: Elipses de incertidumbre [1].

Control en lazo cerrado para un robot móvil

El sistema opera bajo un esquema de Control en lazo cerrado. A diferencia del lazo abierto, este método utiliza la retroalimentación constante de la pose estimada para comparar la posición actual con la meta deseada. Es decir, que utiliza la información del estado actual del robot para ajustar sus acciones y alcanzar un objetivo deseado [4]. A diferencia del lazo abierto este sistema detecta y corrige errores generados por perturbaciones externas, ruido en los sensores, o incertidumbres del modelo como el deslizamiento de las ruedas.

El funcionamiento se basa en un ciclo continuo que puede clasificarse en estas etapas [4, 5, 6]:

1. **Estimación y Percepción de la Pose:** El robot utiliza sensores como el IMU para la orientación y determinar su posición real (x , y , θ) así como también encoders para la velocidad de las ruedas [4]. La `/pose` o `/goals` en este reto se encarga de determinar la referencia global del robot en el plano a partir de la integración discreta de las velocidades lineales del robot las cuales se determinan por el modelo cinemático directo del robot [4].
2. **Cálculo del error:** Se compara la posición real con la trayectoria de referencia (el camino deseado). La diferencia entre ambas se denomina *error de seguimiento*. El error de seguimiento es la medida de la desviación entre el estado actual del robot y el estado deseado definido por la trayectoria. En este

proyecto, el error no se calcula de forma global, sino que se divide en dos componentes polares para facilitar el control del robot diferencial:

- a. *Error de Distancia (Rho)*: Se define como magnitud del vector que une la posición actual del robot (x, y) con la coordenada objetivo (x_goal, y_goal), esto representando cuanto camino lineal le falta recorrer al robot. La fórmula es la siguiente:

$$\rho = \sqrt{(x_goal - x)^2 + (y_goal - y)^2}$$

- b. *Error de Orientación (Alpha)*: Es la diferencia angular entre el ángulo de vista del robot y el ángulo necesario para apuntar directamente hacia el objetivo, este error indica al robot cuánto tiene que girar sobre su eje para lograr el objetivo. La fórmula es la siguiente:

$$\alpha = \text{atan2}(y_goal - y, x_goal - x) - \theta$$

3. **Acción de Control**: Un controlador procesa el error y genera comandos de velocidad para los motores que minimicen la desviación.

PID aplicado a un robot móvil diferencial

La aplicación del controlador en este proyecto se estructura mediante un lazo de control de posición global. A diferencia de los sistemas industriales complejos que controlan cada rueda por separado, este sistema utiliza la retroalimentación de la odometría para corregir el desplazamiento del robot hacia una meta específica en el plano. Para que el Puzzlebot se desplace con precisión, el sistema se enfoca en el siguiente nivel:

Lazo Externo de Seguimiento de Puntos

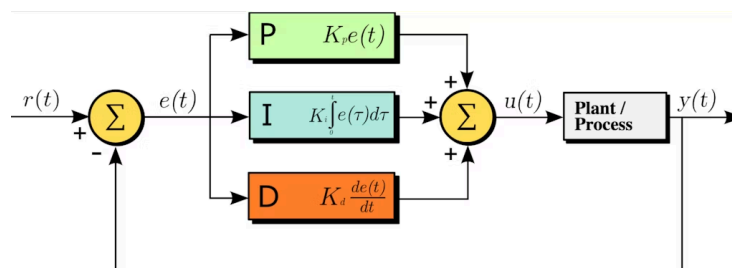


Figura 4: Diseño de un PID. <https://dewesoft.com/es/blog/que-es-un-controlador-pid>

Este lazo utiliza el modelo cinemático del robot para calcular las velocidades lineal (v) y angular (w) necesarias para alcanzar un punto. Compara la pose real estimada (obtenida mediante el nodo de localización y encoders) con la posición de la meta para generar los comandos de velocidad.

En este caso, el PID corrige el error de distancia (ρ) hacia el objetivo:

- **Proporcional (P).** Reduce el error de distancia actual. Si el robot está lejos del punto, aumenta la velocidad lineal proporcionalmente a esa diferencia para acercarse más rápido.
- **Integral (I).** Elimina el error acumulado. Es fundamental para asegurar que el robot llegue exactamente a la coordenada deseada, compensando la fricción que podría detener al robot antes de tiempo. Se implementó con una técnica de clamping para evitar saturación.
- **Derivativo (D).** Anticipa cambios rápidos en la distancia y ayuda a suavizar el frenado cuando el robot se acerca a su valor de referencia, evitando que se pase de la meta.

Robustez de un controlador

La robustez de un controlador se basa principalmente en su capacidad para garantizar el cumplimiento de un objetivo de control, seguir una trayectoria, ante la presencia de perturbaciones externas, ruidos en los sensores e incertidumbres físicas no modeladas [4]. Los principales puntos a tomar en cuenta para un sistema robusto son:

1. **Manejo de Incertidumbres y Perturbaciones.** Un controlador robusto debe ser capaz de minimizar el impacto de factores que no están incluidos en el modelo matemático original, tales como el deslizamiento de las ruedas, no linealidades y ruidos e imperfecciones del entorno [4].
2. **Estrategias de Control Específicas.** Existen algunas técnicas para el diseño que implementan un control robusto como lo son:
 - a. *Control en Modo Deslizante (SMC):* Es una de las técnicas más citadas por su robustez. Se basa en forzar al sistema a permanecer sobre una superficie deslizante predefinida [1]. Una vez que el sistema alcanza este estado, adquiere características invariantes ante perturbaciones.
 - b. *Control de Espacios Nulos (Hierarquización):* Proporciona robustez al establecer una jerarquía de tareas donde la tarea principal, como la posición, se ejecuta sin interferencia de las tareas secundarias, como la orientación, proyectando estas últimas en un espacio que no afecta la precisión del objetivo primordial [4].

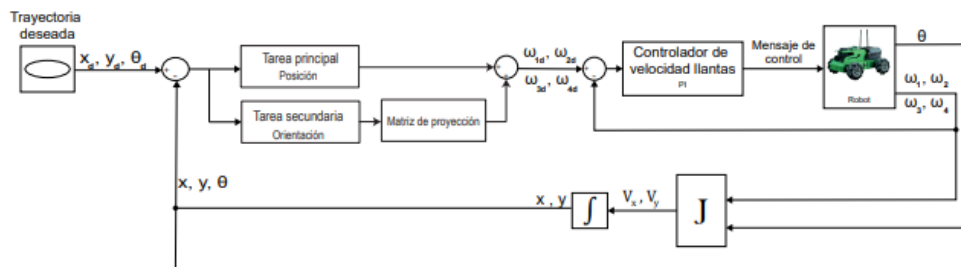


Figura 5: Diseño de un controlador usando lazo de control en espacios nulos [4].

3. **Arquitectura de Lazo Cerrado y Sintonización.** Para que un controlador, como el PID, sea robusto, sus ganancias deben ajustarse cuidadosamente

considerando las perturbaciones y ruidos específicos del entorno de operación. Además, para mejorar la robustez frente a sensores ruidosos, se utilizan observadores, como los basados en la técnica *Super Twisting*, que permiten obtener estimaciones de velocidad más limpias y estables que la simple derivación de la posición [5]. Por último, la implementación de una estructura de doble lazo, con un lazo interno de velocidad, asegura que las ruedas mantengan la consigna deseada incluso ante obstáculos físicos como pendientes, haciendo que el sistema sea resistente a fallos mecánicos menores [4].

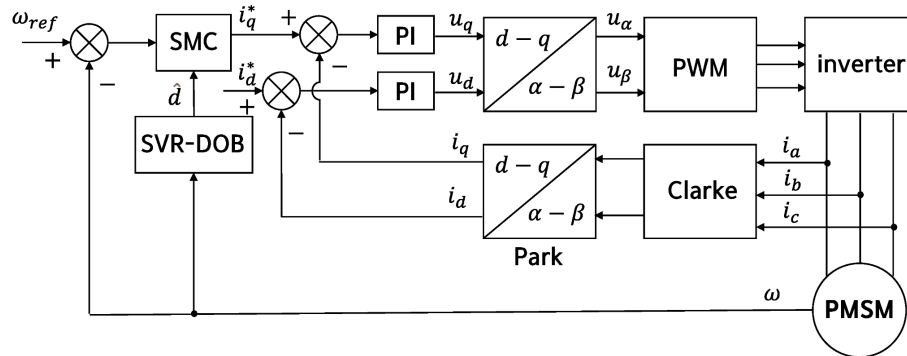


Figura 6: Diseño de un controlador usando la técnica super twisting, PI y SMC para mejorar la robustez. <https://www.mdpi.com/2076-3417/12/21/10749>

Cinemática de robots con accionamiento diferencial

Es la representación matemática que describe cómo el movimiento de sus ruedas independientes se traduce en el desplazamiento y la rotación del vehículo [7]. En el caso del Puzzlebot, se utiliza un sistema de accionamiento diferencial, el cual consiste en dos ruedas motrices independientes situadas en un mismo eje y una rueda loca para brindar estabilidad. Este modelo relaciona las velocidades angulares de cada rueda con la velocidad lineal y angular total del robot basándose en su geometría, específicamente en el radio de las ruedas (r) y la distancia entre ellas (L) [7]. El funcionamiento de este tipo de robots se divide en dos enfoques principales:

- **Cinemática Directa (Forward Kinematics):** Permite calcular la velocidad lineal v_x y la velocidad angular w del robot a partir de las velocidades de rotación medidas de la rueda derecha y de la rueda izquierda
- **Cinemática Inversa (Inverse Kinematics):** Se utiliza para determinar qué velocidades deben tener las ruedas para que el robot alcance una velocidad lineal y angular deseada

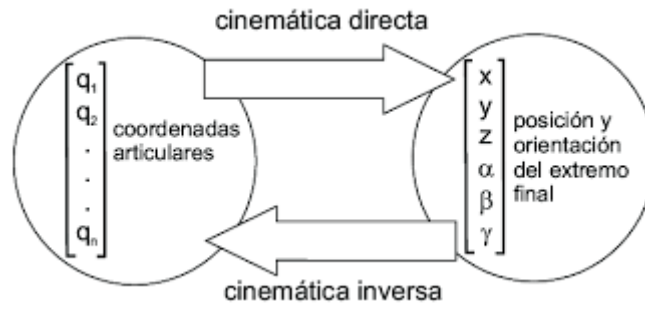


Figura 7: Relacion entre la cinemática directa e inversa

https://www.researchgate.net/figure/Figura-216-Relacion-entre-cinemática-directa-e-inversa_fig15_287995531

El modelo se divide en dos partes:

1. *Cálculo de velocidades lineales y angulares.* La velocidad lineal total v del centro del robot y su velocidad angular w se obtienen mediante las siguientes ecuaciones:

$$\omega = (v_R - v_L)/b \quad V = \omega \cdot R = \frac{v_R + v_L}{2} \quad (1)$$

Donde b es el ancho de la vía o distancia entre las ruedas (wheelbase) [8].

2. *Representación en coordenadas globales.* Para conocer la posición del robot en cada instante, se transforma el movimiento del cuerpo al sistema de coordenadas global utilizando el ángulo de orientación actual ϕ [5]

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\phi} \end{bmatrix} = \begin{bmatrix} \cos \phi & 0 \\ \sin \phi & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} V \\ \omega \end{bmatrix} \quad (2)$$

Aplicación en el controlador

En la implementación de este reto, se enfrenta el problema de la cinemática inversa: dado que el controlador PID genera una velocidad lineal v y una angular w Para alcanzar una meta, es necesario calcular la velocidad que debe tener cada rueda. Esto se hace mediante las ecuaciones:

$$\begin{aligned} v_R &= r \cdot \omega_R \\ v_L &= r \cdot \omega_L \end{aligned} \quad (3)$$

$$\omega_R = \frac{V + \omega \cdot b/2}{r} \quad \omega_L = \frac{V - \omega \cdot b/2}{r} \quad (4,5)$$

Las velocidades se convierten a radianes por segundo (w_{rueda}) dividiendo entre el radio de la rueda (r).

Procesamiento de imágenes en una tarjeta embebida

Se denomina *Visión Embebida* al procesamiento de imágenes en tarjetas embebidas y consiste en integrar algoritmos en hardware dedicado (circuitos integrados, GPUs o FPGAs) que transforman datos visuales en decisiones en tiempo real [9]. El reto principal es ejecutar tareas computacionales pesadas, tales como la detección de bordes, la segmentación y la clasificación de imágenes, en entornos con recursos limitados de memoria y energía [9, 10].

Para implementar estas soluciones, se requiere de un ecosistema que equilibre flexibilidad y potencia, conformado por:

- Software: utilizando la librería de código abierto OpenCV y Python por la gran comunidad que contribuye al desarrollo de aplicaciones [9].
- Sistemas Operativos: Linux es el estándar cuando se trata de estabilidad y seguridad y permite mayor flexibilidad que un sistema operativo tradicional [9].
- Controladores y Sensores: los microcontroladores permiten un rápido prototipado para la interacción con el entorno físico, pero para videos de alta resolución se necesitan módulos más potentes [9].

De este modo, ante la exigencia del procesamiento en tiempo real, la arquitectura del hardware resulta crítica:

- GPUs (Unidades de Procesamiento Gráfico): módulos como los de NVIDIA (ej. Tegra o Cortex A57) son apropiados para ejecutar en paralelo múltiples redes neuronales y procesar video con baja latencia [9, 10].
- FPGAs y DSPs: estas tarjetas permiten un procesamiento en paralelo masivo y directo, a través de hardware, que elimina los cuellos de botellas típicos de los buses seriales convencionales. Esto asegura que la latencia sea de microsegundos [10].
- Optimización de algoritmos: se utilizan técnicas como la cuantización (disminución de la precisión de los datos) y la poda de redes neuronales para que modelos de IA complejos puedan caber en la RAM de la tarjeta [10].

El rendimiento se mide en términos del throughput (píxeles procesados por segundo) y de la latencia; así, la tendencia actual es reducir el ancho de banda mediante el procesamiento selectivo (solo analizar áreas visualmente significativas) y la integración de modelos de deep learning eficientes, permitiendo que las tarjetas embebidas operen de forma autónoma, económica y confiable en sectores como la robótica y el automotriz [10].

Interconexión entre la Jetson y la cámara

La NVIDIA Jetson Nano es la unidad de procesamiento principal del sistema, encargada de tareas de alto nivel como: ejecutar ROS 2, procesar datos, implementar algoritmos complejos, visión por computadora, etc. Cuenta con mayor capacidad de cómputo que la Hackerboard, lo que permite ejecutar procesos más complejos aunque con menor prioridad en tiempo real. La Raspberry Pi Camera es el sensor visual del Puzzlebot utilizado principalmente para: capturar imágenes y video, aplicaciones de visión por computadora, detección de objetos, etc. [11]

La Jetson Nano permite usar cámaras de alta velocidad gracias a su soporte nativo para el protocolo MIPI-CSI (Mobile Industry Processor Interface - Camera Serial Interface), que conecta la cámara directamente al procesador sin la latencia del USB. A diferencia de los modelos anteriores que requieren configuraciones complejas de drivers, la Jetson Nano viene con un conector compatible y drivers preinstalados para la Raspberry Pi Camera V2 (sensor Sony IMX219). Esto permite conectar directamente a la Jetson Nano mediante interfaz CSI, permitiendo el procesamiento en tiempo real de las imágenes [12].



Figura 8: Conexión física de la Jetson Nano con la Raspberry Pi Camera V2.
https://cdn1.botland.store/img/art/inne/15751_5a.jpg

Visión por computadora

La visión por computadora es un conjunto de técnicas que permiten extraer información de imágenes, video o nubes de puntos, e incluye técnicas basadas en *deep learning*, técnicas basadas en detección y extracción de características, técnicas de procesamiento de nubes de puntos, técnicas de procesamiento de visión en 3D, y, el procesamiento de imágenes que suele aplicarse como preprocesamiento en la visión por computadora y que comprende técnicas como aumento de datos, conversión o corrección de color, redimensionamiento de imágenes, eliminación de ruido, entre otras [13].

La visión por computadora es importante para una amplia gama de aplicaciones del mundo real, incluyendo a los sistemas autónomos que utilizan varios sensores que recopilan datos visuales del entorno, los cuales son utilizados para segmentar carreteras, senderos o edificios, así como para detectar personas y vehículos [13].

Métodos de procesamiento de imágenes y visión por computadora

- **Espacios de color y conversión.** Los espacios de color son organizaciones específicas que determinan cómo los colores de las imágenes digitales son representados y procesadas en visión computacional, y son esenciales para aplicaciones como la detección de objetos, segmentación, corrección de color y compresión; un espacio de color define el modelo de color y la función de mapeo abstracto utilizada para definir los colores reales [14].

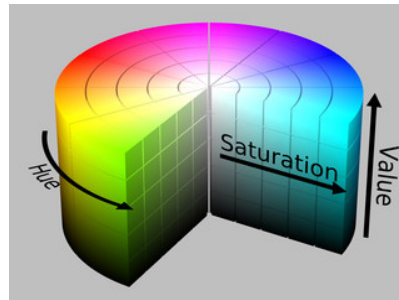


Figura 9: Representación tridimensional del espacio de color HSV [14].

En OpenCV, la conversión de espacios de color se realiza con la función `cvtColor()`, que permite la transformación de imágenes entre espacios de color como RGB y HSV. El espacio RGB es el más utilizado comúnmente, mientras que otros como el HSV (Hue, Saturation, Value) permite detectar y filtrar colores con mayor facilidad, incluso cuando la iluminación cambia, ya que es más cercano a cómo los humanos representan los colores [14].

- **Ajuste de brillo y contraste.** Ajustar el brillo y contraste de una imagen puede ayudar a corregir defectos en las imágenes y facilita la visualización de detalles. Existen diversas maneras de ajustar el brillo y contraste con OpenCV, dos de ellas son las funciones `addWeighted()` y `convertScaleAbs()`, que ajustan el brillo al añadir un valor escalar a cada pixel, y el contraste al escalar el valor de los pixeles [15].

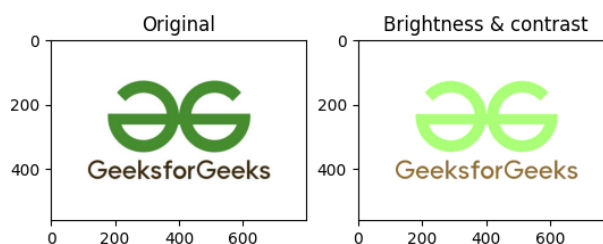


Figura 10: Comparación antes y después del ajuste de brillo y contraste [15].

- **Desenfoque Gaussiano.** El desenfoque Gaussiano es una técnica de filtrado que utiliza una función gaussiana para crear un efecto de suavizado en una imagen, este suavizado reduce el ruido de la imagen y el detalle. En OpenCV, puede utilizarse con la función `GaussianBlur()`, que toma parámetros como la imagen, el tamaño del kernel, la desviación estándar para las direcciones X y Y, y `borderType` [16].

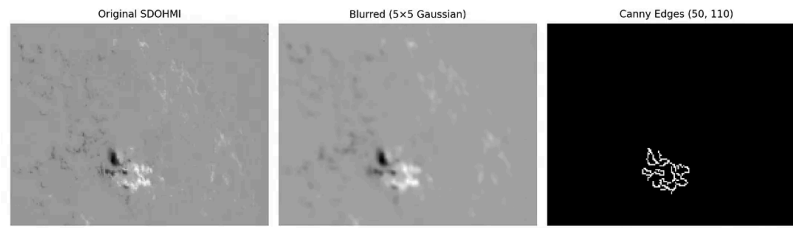


Figura 11: Comparación sobre el uso del desenfoque Gaussiano [16].

- **Detección de blobs (regiones).** La detección de blobs es una técnica que permite detectar blobs, que son regiones de una imágenes con propiedades específicas tales como intensidad o color uniformes o dentro de un rango definido; es decir, grupos de píxeles conectados entre sí que se consideran una unidad de información. En OpenCV, la detección de blobs se realiza mediante `SimpleBlobDetector`, que es un algoritmo que sigue los siguiente pasos: umbralización, agrupación, unión (merging) y cálculo del centro y del radio. Las técnicas para detección de blobs incluyen el Laplaciano de Gauss (LoG), Diferencia de Gaussianas (DoG) y Determinante de Hessiano (DoH) [17].

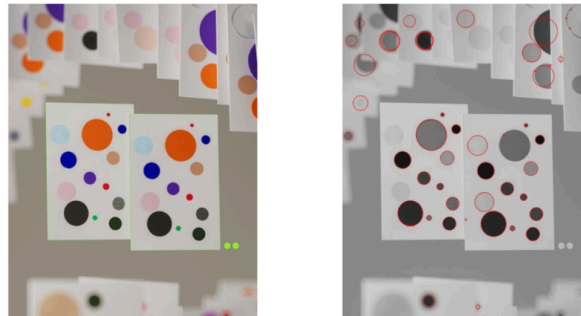


Figura 12: Ejemplo de la detección de blobs [17].

En OpenCV, los parámetros determinan los filtros para la detección de blobs. Entre estos se encuentra detección por: color, tamaño, forma, circularidad, convexidad y radio inercial. Entre estos, el filtrado por color determina si se filtran los blobs oscuros o claros, y el filtrado por circularidad utiliza una medición que determina qué tan cercano es el blob a un círculo [17].

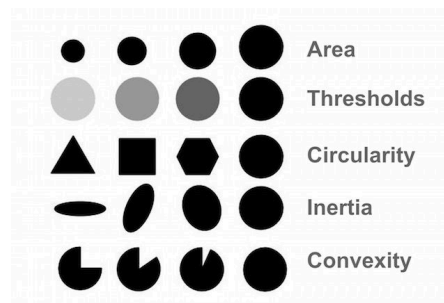


Figura 13: Ilustración de los parámetros del detector de blobs [17].

- **Operaciones morfológicas.** Las operaciones morfológicas son un conjunto extenso de operaciones que se utilizan para procesar imágenes con base en figuras geométricas. Las operaciones morfológicas, al implementar un elemento estructurante a una imagen de entrada, generan otra imagen de salida que tiene el

mismo tamaño. La erosión y la dilatación son las operaciones morfológicas más elementales. La erosión elimina píxeles de los bordes de los objetos, a la vez que la dilatación añade píxeles a dichos límites. La cantidad de píxeles que se eliminan o se añaden a los objetos está determinada por la forma y el tamaño del elemento estructurante empleado para el procesamiento [18].

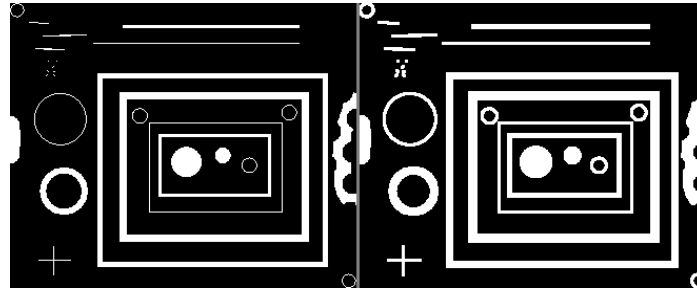


Figura 14: Ejemplo de la dilatación, imagen original y procesada [18].

Robustez en sistemas de procesamiento de imágenes

La robustez en la visión artificial se define como la capacidad de un algoritmo para mantener su integridad operativa y precisión estadística ante la presencia de perturbaciones externas y variaciones en el entorno de captura. En el contexto de la robótica autónoma, este concepto es fundamental, ya que los sensores visuales están intrínsecamente sujetos a ruidos térmicos, cambios abruptos en la iluminación y oclusiones dinámicas [19].

Uno de los pilares de la robustez visual es la selección de un espacio de color que minimice la dependencia de la intensidad de la luz. Mientras que el modelo RGB es dependiente de la luminancia, el espacio HSV (Hue, Saturation, Value) permite una separación efectiva entre la información cromática (matiz) y la iluminación (brillo). Según Gonzalez y Woods (2018), esta propiedad es esencial para el diseño de clasificadores robustos, ya que el matiz permanece relativamente constante bajo diferentes condiciones de exposición, facilitando una segmentación de objetos más fiable en entornos no controlados [20].

Más allá del color, la robustez se refuerza mediante el análisis de la morfología geométrica. La implementación de filtros de circularidad, área y excentricidad permite al sistema discriminar entre el objeto de interés y el ruido ambiental. La teoría de análisis estructural sugiere que el uso de descriptores de forma basados en momentos de Hu o circularidad de compactación permite una identificación invariante a la escala y la rotación [21]. Estos filtros actúan como una capa de validación que asegura que las detecciones erráticas sean descartadas antes de ingresar a la lógica de control. Por otro lado, la robustez también está ligada a la eficiencia computacional. Un sistema robusto debe equilibrar la complejidad del procesamiento con la latencia del sistema de control. La integración de técnicas de filtrado morfológico, como la erosión y dilatación, ayuda a eliminar el ruido tipo "sal y pimienta" como menciona corke, permitiendo que la señal visual que alimenta al sistema de navegación sea limpia y persistente en el tiempo [22].

Esto es crítico para evitar comportamientos inestables en la navegación reactiva, donde una detección falsa podría traducirse en una respuesta motriz errónea.

Solución del problema

1. Estimación de pose

En este reto, la posición del robot ya no se estimaba dependiendo del tiempo de ejecución dada ya sea la velocidad o tiempo en el que debía hacer la trayectoria, sino que se implementó un nodo de localización. Este es el encargado de recibir la retroalimentación de los encoders para resolver el problema de la odometría.

Se configuraron dos suscriptores a los tópicos `VelocityEncR` y `VelocityEncL` los cuales son tópicos propios del Puzzlebot que publican información de los encoders. En este nodo también se utilizó una política de QoS (Quality of Service) de tipo `Best Effort` que garantiza recibir los datos aunque haya latencia en la que se pueden perder los paquetes.

Se utilizaron las constantes físicas del Puzzlebot como el radio de la rueda $r = 0.05m$ y la distancia entre ruedas $l = 0.18m$ las cuales se implementaron en las ecuaciones de espacio de estado para transformar las velocidades angulares de las ruedas en velocidades lineales y angulares del chasis.

$$v = r \frac{\omega_R + \omega_L}{2}, \quad \omega = r \frac{\omega_R - \omega_L}{l}$$

Después, se necesitó utilizar la integración de Euler con un diferencial de tiempo Δt que se calculaba constantemente mediante el reloj del sistema de ROS 2. Esto permite obtener a partir de la velocidad, la estimación de la pose (x, y, θ) . Al usar el reloj de ROS 2, permite que la estimación sea mejor a pesar de que haya variaciones del procesamiento del CPU.

Para cumplir con el estándar del tipo de mensajes que maneja ROS 2 para los datos de odometría que son de tipo `nav_msgs`, la orientación se convirtió de ángulos de Euler a cuaterniones con la librería de `transforms3d`, publicando el mensaje completo al tópico `/odom`.

2. Generador de trayectorias

El nodo generador de trayectorias se encarga de coordinar la progresión de los puntos a los que debe llegar el robot o waypoints mediante el monitoreo constante de la posición suscribiéndose al tópico de `/odom` del nodo de localización.

El usuario puede definir la trayectoria deseada a través del número de puntos y sus coordenadas (x, y) ; después de definir la trayectoria, el nodo calcula constantemente el error de posición actual, definido como la distancia euclidiana (ρ) entre la ubicación del robot y el objetivo. Para optimizar el movimiento y evitar que el robot se detenga, se

implementó un umbral de aceptación de 0.05 metros para los puntos intermedios. Una vez que el robot ha ingresado en ese rango de tolerancia, el sistema actualiza el índice del objetivo y publica el siguiente destino.

La información que se le transfiere al nodo controlador se realiza mediante un mensaje personalizado llamado `GoalList`. Este mensaje tiene la estructura para que el control a lazo cerrado espere la información de la siguiente manera:

`current_goal` es el que especifica la coordenada exacta hacia la cual el controlador debe dirigir los esfuerzos del motor y orientación.

`next_goal` es el que proporciona la ubicación del siguiente vértice, permitiendo al sistema tener cambios de dirección.

`final_goal` usa una bandera booleana que se activa solo al procesar el último vértice de la trayectoria, indicando al controlador que debe detener la trayectoria de forma inmediata para evitar errores acumulativos que afecten el control.

3. Detección de semáforo con visión computacional

Se desarrolló un nodo de ROS 2 llamado `TrafficLightNode` diseñado para detectar el color de un semáforo (rojo, amarillo, verde) utilizando visión computacional con OpenCV. El nodo tiene un suscriptor al tópico `/video_source/raw` y tiene dos publicadores a los tópicos `/semaforo_estado` e `/image_processing/result`.

El nodo utiliza `SimpleBlobDetector` de OpenCV para filtrar el área correspondiente a la luz del semáforo, el cual se configuró para buscar blobs circulares con utilizando `filterByCircularity` con una circularidad mínima de 0.72, de color claro sobre un fondo oscuro, lo cual corresponde a las máscaras de color. Adicionalmente se utilizó un rango de píxeles de 50 a 50000 para filtrar los blobs por área, filtrando ruidos pequeños y poder detectar luces a distancia.

El procesamiento de la imagen se realiza en la función `camera_callback`, correspondiente al suscriptor; de este modo cada que llega una imagen de la cámara mediante el tópico `/video_source/raw`, el código comienza por convertir la imagen del mensaje tipo `Image` de ROS 2 a un formato `bgr8` para OpenCV, con el método `imgmsg_to_cv2` de `CvBridge`.

Dentro del callback, el preprocesamiento comienza con una reducción del brillo y un desenfoque gaussiano para suavizar el ruido; posteriormente, se cambia el espacio de color de BGR a HSV, para detectar colores específicos a través de la creación de máscaras binarias, donde blanco representa el color detectado y el negro lo demás.

Las máscaras binarias se crean utilizando la función de OpenCV `inRange()`, centrándose principalmente en los valores del Hue, que indica el color a detectar; además, como caso particular, el rojo se define en dos rangos debido a que está en

ambos extremos del espectro HSV. Como máscara auxiliar, se crea una máscara que detecta el centro blanco brillante de la luz, el cual es dilatado para crear un anillo alrededor. Luego, se verifica si ese anillo coincide con el color esperado, lo que ayuda a verificar que lo que brilla sea la luz del semáforo.

Una vez obtenidas las máscaras, se buscan blobs en las máscaras de rojo, amarillo y verde, filtrando adicionalmente por área y circularidad. De los blobs detectados se selecciona el color que tenga el blob más grande, y posteriormente se aplica un umbral con valor de 80, que permite que la detección del semáforo se aplique cuando la distancia respecto al robot sea de 30 cm o menos.

Finalmente, se publica en el tópic `/semaforo_estado`, un mensaje tipo String con la letra del estado final: R, A, V, o N cuando no se detectó el semáforo o está a una distancia mayor a 30 cm. Adicionalmente, en el tópic `/image_processing/result` se publica una imagen procesada donde se dibuja un círculo sobre la luz detectada, así como el tamaño del blob, lo que resulta útil para calibración en tiempo real y debugging.

4. Controlador de lazo cerrado

El controlador implementa un algoritmo reactivo que responde ante las lecturas de los encoders y al estado del semáforo, lo que permite que el robot tenga una respuesta inmediata ante su posición real en el espacio y su entorno.

Primero para que la navegación esté optimizada, el controlador hace una conversión de coordenadas cartesianas que son las que provienen del nodo que publica la odometría, hacia un sistema de coordenadas polares que es relativo al punto de destino.

En este caso ρ es la hipotenusa del triángulo formado por la distancia en X y Y. Representa la distancia lineal que le falta al robot para llegar al punto objetivo. Este se calcula de la siguiente manera:

$$\rho = \sqrt{\Delta x^2 + \Delta y^2}$$

Después se calcula α que es el error de orientación. Primero se calcula el ángulo hacia el objetivo usando `atan2` y luego se resta la orientación actual del robot. Este resultado es cuánto debe girar el robot.

$$\alpha = \text{atan2}(\Delta y, \Delta x) - \theta$$

Se implementó un controlador PID en cascada para la velocidad lineal. A diferencia de un PID simple, este sistema utiliza un lazo exterior que procesa el error de distancia ρ para determinar la velocidad deseada, y un lazo interior que corrige la velocidad real del robot comparándola con dicha diferencia. El controlador genera una velocidad proporcional a la proximidad del objetivo, integrando los tres componentes del PID para asegurar que el Puzzlebot llegue a la meta con precisión, compensando

errores acumulados y suavizando el movimiento antes de detenerse. Para la parte angular, se utiliza un control proporcional que ajusta el giro del robot basándose en el error α , permitiendo que el robot se oriente correctamente mientras avanza.

5. Implementación y Sintonización

Los pasos técnicos seguidos en este reto son:

1. **Cálculo Cinemático:** Se utiliza la cinemática diferencial para estimar la posición del robot y determinar el ángulo hacia la meta.
2. **Lectura de Sensores:** Se utilizan los encoders del Puzzlebot para obtener la velocidad de cada rueda, la cual se integra en el nodo de localización para conocer la posición actual (x , y).
3. **Sintonización de Parámetros:** No se utilizó Ziegler-Nichols. Las constantes del controlador (K_p , K_i , K_d) se determinaron mediante un método experimental de prueba y error, buscando un equilibrio entre rapidez de respuesta y suavidad, ajustando los valores hasta obtener un movimiento estable y preciso.
4. **Discretización:** El controlador se implementó de forma digital en un nodo de Python dentro de ROS 2, calculando el error y la salida en cada paso de tiempo (Δt) medido por el reloj del sistema.
5. **Control de Orientación:** Para corregir el error de orientación α , se aplicó un simple control proporcional junto con la función `wrap_to_pi` la cual restringe las rotaciones a un rango de $-\pi$, π para asegurarse de que los giros sean más óptimos y no dar vueltas innecesarias.

Finalmente, aplicando el control PID en cascada para la velocidad lineal y el control P para la rotación, la lógica de control es la siguiente:

1. Si el error angular (α) hacia el objetivo es mayor de 0.15 rad , el sistema entra en fase de rotación, priorizando el giro sobre su propio eje.
2. Una vez que está alineado, el control PID exterior calcula la velocidad requerida y el PID interior ajusta la potencia de los motores, mientras que el control angular realiza ajustes mínimos simultáneos para mantener la trayectoria.
3. El robot reacciona al tópico `/semaforo_estado`. Si detecta luz roja, se detiene totalmente; si detecta amarilla, reduce su velocidad mediante un multiplicador de 0.7; y con verde, retoma su marcha normal; Además, después de haber detectado una luz roja, el robot permanece detenido hasta ver una luz verde.
4. Una vez que la distancia es menor al umbral de tolerancia que está definido como 0.05m , el error de posición se considera que ha sido mitigado. Cuando se cumple esta condición, el sistema valida la llegada del robot al destino y el generador de trayectorias procede a publicar el siguiente punto.

6. Manejo de no-linealidades

Para que el controlador PID fuera lo más preciso posible, se aplicaron algunas estrategias para que el sistema fuera lo más robusto posible.

Se utilizaron valores de zona muerta y saturación $v_{min} = 0.05m/s$ y $v_{max} = 0.3m/s$ para limitar la salida del PID evitando que el controlador demande velocidades que supere la velocidad máxima a la que puede alcanzar o velocidades que no superan la inercia inicial. Por el otro lado para la velocidad angular se utilizaron valores de zona muerta y saturación $v_{min} = 0.1$ y $v_{max} = 0.5$

Para los puntos intermedios de la trayectoria que debe realizar el robot, se estableció una tolerancia dinámica la cual es de 5 cm para permitir un movimiento fluido sin que se detenga totalmente en cada esquina, mientras que para el punto final es de 2 cm para que pueda cerrar de manera exacta la trayectoria.; además, se limpia el error acumulado del PID para un cierre preciso.

Por último, el proceso de sintonización del controlador PID se realizó mediante el método heurístico:

1. Se incrementó K_p hasta observar una respuesta deseada y rápida
2. Se añadió K_d para suavizar las detenciones en las esquinas y eliminar oscilaciones
3. Se ajustó K_i mínimamente para corregir desvíos laterales constantes

7. Estrategias de robustez implementadas

Para asegurar la robustez del Puzzlebot en este challenge, se aplicaron las siguientes estrategias dentro del código:

1. *Anti-Windup/Clamping*. En el término integral del PID, se implementó una técnica de limitación o clamping. Esto evita que el error acumulado crezca de forma desmedida cuando el robot encuentra un obstáculo físico o una saturación en los motores, previniendo oscilaciones violentas al recuperar el movimiento.
2. *Manejo de Zonas Muertas (Deadzone)*. El controlador incluye un límite mínimo de salida (`min_out`). Esto asegura que el voltaje enviado a los motores sea siempre suficiente para vencer la fricción estática, permitiendo que el robot se mueva incluso ante errores muy pequeños.
3. *Filtrado de Orientación con wrap_to_pi*. Para evitar giros innecesarios o bruscos, se utilizó una función que normaliza el error angular entre $-\pi$, π . Esto garantiza que el controlador siempre elija la ruta de giro más corta, aportando estabilidad a la navegación.
4. *Tolerancia de Error (Threshold)*. Se estableció un umbral de precisión de 5 cm para considerar una meta como alcanzada. Esta tolerancia permite que el sistema sea inmune al ruido de baja amplitud de los encoders, evitando que el robot realice micro-correcciones infinitas en el punto de destino.

Resultados

Después de haber realizado el algoritmo de detección de semáforo con los valores adecuados para cada color en el espacio HSV, se realizaron varias pruebas para saber a qué distancia y con qué intensidad se detectan correctamente los colores. Para aquello, se ejecutó el nodo de `traffic_light_node2` el cual publica la imagen procesada al tópico `/image_processing/result`. Se puede suscribirse a este tópico mediante la herramienta de `rqt_image_view` para visualizar la imagen procesada. En esta se puede ver que dependiendo del color del semáforo, se enciende la luz con un círculo del mismo color y además, en la parte superior izquierda se observa un texto que indica el carácter que se está publicando al tópico de `/semaforo_estado`. A continuación se muestra el funcionamiento de la detección de color del semáforo con los 3 colores:

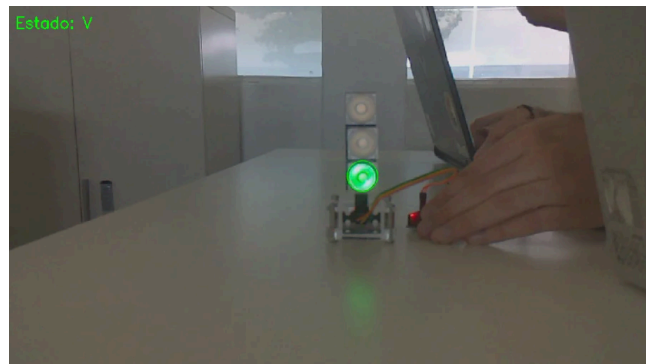


Figura 15: Detección del color verde.

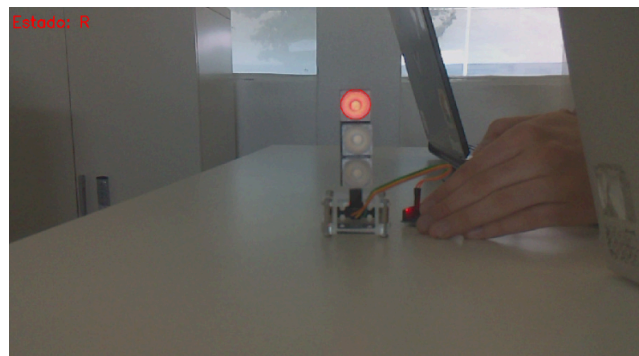


Figura 16: Detección del color rojo.

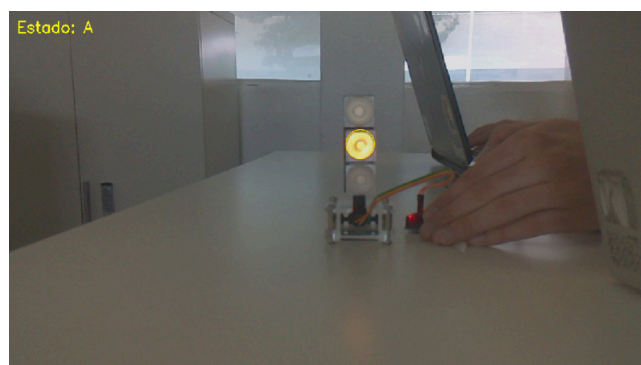


Figura 17: Detección del color amarillo.

Después de haber verificado que los colores son detectados correctamente a distintas distancias y con distinta iluminación, se muestran los resultados de cómo se implementa la medición de la distancia del semáforo con respecto al tamaño del blob. Se observa cómo al ser un valor mayor al umbral de 80, el carácter que indica el color es publicado.

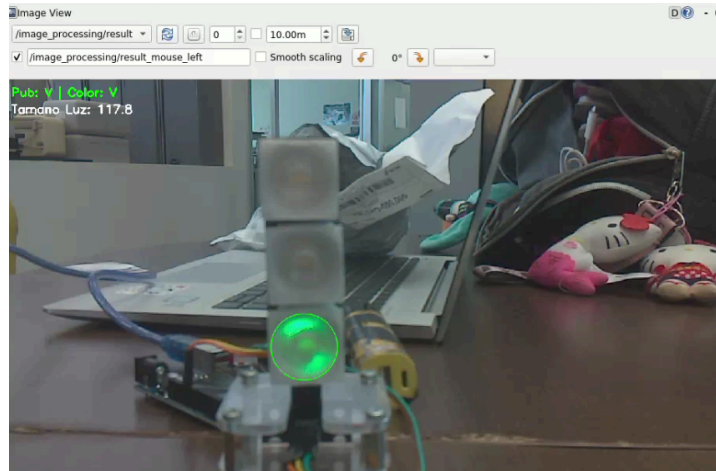


Figura 18: Visualización del tamaño del blob del círculo verde.

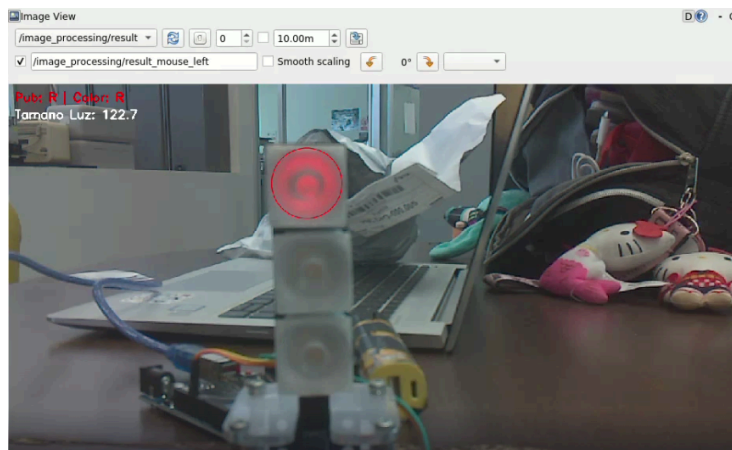


Figura 19: Visualización del tamaño del blob del círculo rojo.

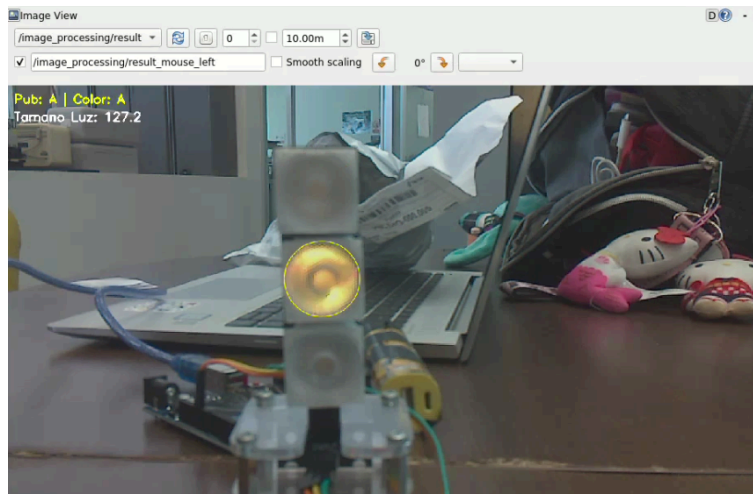


Figura 20: Visualización del tamaño del blob del círculo amarillo.

Se realizaron distintas pruebas del sistema utilizando el Puzzlebot con el objetivo de validar el correcto funcionamiento de este. Inicialmente se probó realizando una trayectoria en línea recta la cual estaba definida en el nodo de `path_generator` con el objetivo de llegar al punto $[2, 0]$.

De igual manera se implementó la visualización gráfica de las trayectorias en tiempo real mientras el robot se movía. Esto funcionó para saber qué tan exacto seguía la trayectoria y si se tenía que modificar algún valor del controlador PID. A su vez, para verificar el funcionamiento de la lógica de estados de detección de color, se colocó el semáforo en distintos puntos de la trayectoria mostrando un color diferente.

Teniendo este sistema de interacción con el usuario, se establecieron las siguientes trayectorias:

1. **Línea:** La primera fase de pruebas consistió en realizar una trayectoria en línea recta de dos metros. Como se puede observar en la gráfica de la figura 21, la línea la sigue con bastante precisión tendiendo sólo ligeras oscilaciones. Durante esta prueba, se colocó el semáforo en color rojo y se comprobó que se detuviera de manera exitosa. Al estar detenido, únicamente avanzó cuando el semáforo mostró el color verde.

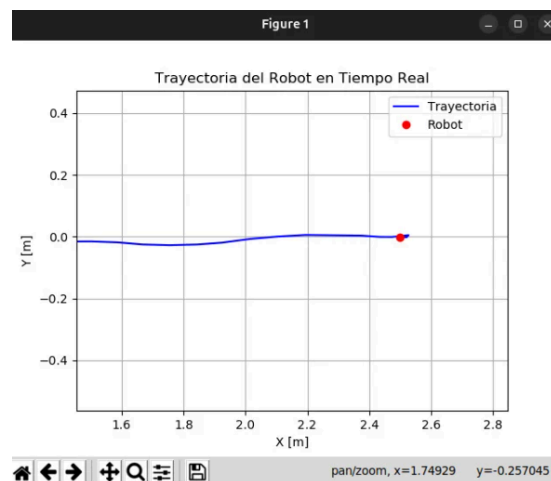


Figura 21: Gráfica de la trayectoria en línea recta.

Link del video demostrativo:

<https://drive.google.com/file/d/11t9Nth84CGqWcPBDbdPI4AR03akecSKX/view?usp=sharing>

2. **Rombo:** En la figura 22 se presenta la trayectoria generada en tiempo real a partir de la secuencia de puntos de consigna: $(0.5, 0.5)$, $(0, 1)$, $(-0.5, 0.5)$ y $(0, 0)$. Los resultados experimentales demuestran que el Puzzlebot cumple con el recorrido de manera satisfactoria, manteniendo un control preciso sobre las velocidades lineal y angular en todo momento. Se destaca la capacidad del sistema para transitar entre coordenadas con cambios de dirección constantes, validando la eficacia del controlador de lazo cerrado y el algoritmo de seguimiento de metas ante trayectorias que exigen maniobras en

múltiples cuadrantes del plano. A su vez se comprobó casi al final de la trayectoria como al mostrar la luz amarilla, su velocidad se reduce y espera a que reciba el color rojo para detenerse.

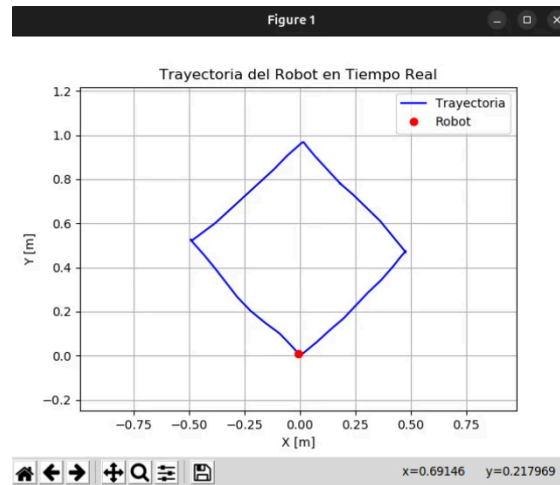


Figura 22: Gráfica de la trayectoria del rombo.

Link del video demostrativo:

<https://drive.google.com/file/d/1Gz-hoIVzNPXmXw104imOICqB-5pTndd5/view?usp=sharing>

3. **Trapezio.** En la figura 23 se presenta la trayectoria generada en tiempo real a partir de la secuencia de puntos de consigna: $(1.0, 0.0)$, $(0.5, 1)$, $(-0.4, 1)$, $(-1.0, 0)$ y $(0, 0)$. Esta prueba permitió evaluar el desempeño del sistema ante una ruta no convencional que requiere desplazamientos en ángulos agudos y cambios de dirección constantes. Los resultados muestran que el Puzzlebot navega entre los waypoints de manera fluida, validando la precisión del nodo de localización y la robustez del controlador PID. De igual manera durante la prueba se comprobó como al leer un color, ya sea amarillo o rojo los cuales modifican la velocidad, a pesar de que se quite el semáforo no cambiarán de estado hasta que cambie a color verde en el caso de ser rojo o cambie a rojo o verde en el caso de ser amarillo.

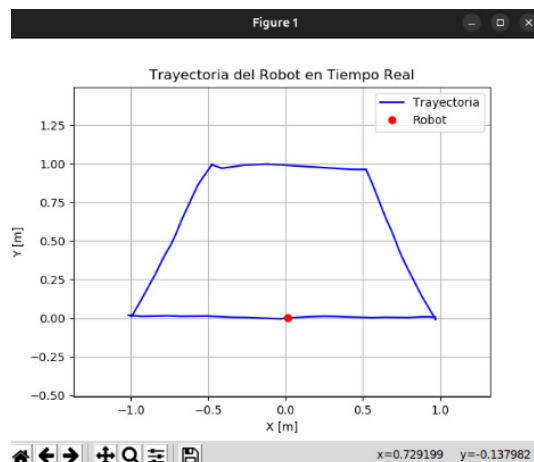


Figura 23: Gráfica de la trayectoria del trapecio.

Link del video demostrativo:

<https://drive.google.com/file/d/1LdcpiLqprapNNF0q-eADjrRhuSes4uET/view?usp=sharing>

Aquí se muestran los videos con distintas trayectorias:
<https://drive.google.com/drive/folders/1VTHvD5ZLpXZUHTKTKw4-DiP4NI2jUQGI?usp=sharing>

Conclusiones

El presente reto permitió validar la eficacia de un sistema de navegación en lazo cerrado frente a los métodos convencionales de lazo abierto. La implementación de una arquitectura de control en cascada (PID) y el uso de retroalimentación constante mediante odometría permitieron al Puzzlebot ejecutar trayectorias con una precisión significativamente mayor, eliminando la dependencia de temporizadores y adaptándose activamente a su posición real. Asimismo, la integración del sistema de visión artificial dotó al robot de una capacidad de toma de decisiones autónoma, permitiéndole interactuar con elementos externos como semáforos para regular su velocidad y asegurar una navegación segura. Como trabajo futuro, se concluye que la robustez del sistema podría incrementarse mediante la fusión de sensores adicionales, como una unidad de medición inercial (IMU) o el uso de visión computacional para la corrección de posición (SLAM), mitigando así las limitaciones de la odometría pura y optimizando la precisión en entornos con superficies irregulares.

Referencias

- [1] Ramírez Benavides, K. (s. f.). Odometría [Diapositivas de Powerpoint]. Recuperado 17 de abril de 2026, de <https://www.kramirez.net/Robotica/Material/Presentaciones/Odometria.pdf>
- [2] Blade. (2024, 2 octubre). *Understanding Odometry and Its Applications*. News - SparkFun Electronics. <https://news.sparkfun.com/11361>
- [3] Dawkins, P. (2022, 16 noviembre). *Differential equations - Euler's Method*. Pauls Online Math Notes. <https://tutorial.math.lamar.edu/classes/de/eulersmethod.aspx>
- [4] López Doicela, R. S. (2025). Estrategias de control, estimación y evasión de obstáculos para un robot móvil omnidireccional con ruedas Mecanum: Desarrollo de un controlador cinemático de seguimiento de trayectorias [Trabajo de Integración Curricular, Escuela Politécnica Nacional]. Repositorio Institucional EPN.
- [5] Sáenz, A., Santibáñez, V., Bugarín, E., Dzul, A., & Ríos, H. (2017). Control de velocidad por voltaje calculado de un robot móvil omnidireccional con realimentación visual. En Memorias del XIX Congreso Mexicano de Robótica (COMRob 2017). Mazatlán, Sinaloa, México: Asociación Mexicana de Robótica e Industria AC.
- [6] Delgado-Báez, J. A., Castro-Linares, R., & Velasco-Villa, M. (2014). Formación de robots móviles omnidireccionales considerando su modelo dinámico. En Memorias del XVI Congreso Latinoamericano de Control Automático (CLCA 2014) (pp. 14-17). Cancún, Quintana Roo, México.
- [7] Dellaert, F., & Sethi, S. (s. f.). *Differential drive robot actions*. Robotics from first principles. https://www.roboticsbook.org/S52_diffdrive_actions.html

- [8] Klančar, G., Zdešar, A., Blažič, S., & Škrjanc, I. (Eds.). (2017). Wheeled mobile robotics: From fundamentals towards autonomous systems. Butterworth-Heinemann
- [9] Vinke, N. (2022, 2 enero). *Image Processing on Embedded Platforms*. Medium.
<https://medium.com/@narayanivinke4545/image-processing-on-embedded-platforms-a460cd6c8a04>
- [10] Boufaïda, S. O., Benmachiche, A., & Maatallah, M. (2026). Real-Time Image Processing Algorithms for Embedded Systems. *arXiv (Cornell University)*.
<https://doi.org/10.48550/arxiv.2601.06243>
- [11] Raspberry Pi Foundation. (s.f.). Getting started with the Camera Module. Raspberry Pi Projects. Recuperado el 5 de abril de 2026, de
<https://projects.raspberrypi.org/en/projects/getting-started-with-picamera/1>
- [12] Kangalov. (2019, 2 abril). *Jetson Nano + Raspberry Pi Camera*. JetsonHacks.
<https://jetsonhacks.com/2019/04/02/jetson-nano-raspberry-pi-camera/>
- [13] Vemulapalli, K. (2025, 20 marzo). *Computer Vision*. MathWorks.
<https://la.mathworks.com/discovery/computer-vision.html>
- [14] Moukthika. (2025a, 29 abril). *Color spaces in OpenCV*. OpenCV.
<https://opencv.org/color-spaces-in-opencv/>
- [15] GeeksforGeeks. (2025, 28 abril). *Image Enhancement Techniques using OpenCV Python*. GeeksforGeeks.
<https://www.geeksforgeeks.org/machine-learning/image-enhancement-techniques-using-opencv-python/>
- [16] Trital, P. (2025, 19 mayo). *Gaussian Blur*. Medium.
https://medium.com/@prajun_t/gaussian-blur-cceb6c76a009

- [17] Moukthika. (2025b, mayo 7). *Blob Detection using OpenCV*. OpenCV.
<https://opencv.org/blob-detection-using-opencv/>
- [18] MathWorks. (s. f.). *Tipos de operaciones morfológicas*. Centro de Ayuda de MATLAB. Recuperado 10 de mayo de 2026, de
<https://la.mathworks.com/help/images/morphological-dilation-and-erosion.html>
- [19] Computer Vision: Algorithms and Applications, 2nd ed. (s. f.).
<https://szeliski.org/Book/>
- [20] Gonzalez, R. C., & Woods, R. E. (2018). Digital Image Processing. Pearson.
- [21] Bradski, G., & Kaehler, A. (2008). Learning OpenCV: Computer vision with the OpenCV library. O'Reilly Media.
- [22] Corke, P. (2023). Robotics, Vision and Control: Fundamental Algorithms in Python. Springer.